

BLITZ

MUST-HAVE

BLITZ

50 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК senior FE blitz round

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	JS CORE CLOSURE	===	HOIST	THIS	ARROW	JS CORE CLOSURE	===	HOIST	THIS	ARROW	
	Q1 · Q2 · Q3 · Q4 · Q5 · Q6 · Q7 · Q8										
ДЕНЬ 2 СРЕДНИЙ	EVENT LOOP NC	PROMISE	MICROTASK			ASYNC	EVENT LOOP	PROMISE	MICROTASK	ASYNC	
	Q9 · Q10 · Q11 · Q12 · Q13 · Q14 · Q15										
ДЕНЬ 3 СРЕДНИЙ	REACT LBACK	KEYS	LIFTING	MEMO	USECAL	REACT LBACK	KEYS	LIFTING	MEMO	USECAL	
	Q16 · Q17 · Q18 · Q19 · Q20 · Q21 · Q22										
ДЕНЬ 4 СРЕДНИЙ	HOOKS	USEEFFECT	USEREF	CONTEXT		HOOKS	USEEFFECT	USEREF	CONTEXT		
	Q23 · Q24 · Q25 · Q26 · Q27 · Q28 · Q29										
ДЕНЬ 5 ТЯЖЁЛЫЙ	BROWSER P	CORS	COOKIE	STORAGE	CS	BROWSER P	CORS	COOKIE	STORAGE	CS	
	Q30 · Q31 · Q32 · Q33 · Q34 · Q35 · Q36										
ДЕНЬ 6 ТЯЖЁЛЫЙ	PERF	LCP	INP	CLS	BUNDLE		PERF	LCP	INP	CLS	BUNDLE
	Q37 · Q38 · Q39 · Q40 · Q41 · Q42 · Q43										
ДЕНЬ 7 СРЕДНИЙ	TS	GIT	TOOLS	A11Y		TS	GIT	TOOLS	A11Y		
	Q44 · Q45 · Q46 · Q47 · Q48 · Q49 · Q50										

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 – БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1

НАГРУЗКА

СРЕДНИЙ

8 ВОПРОСОВ

JS CORE === HOIST THIS ARROW CLOSURE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

JS CORE === HOIST THIS ARROW CLOSURE

ВОПРОСЫ ДНЯ

Q01 == vs ===

Q02 Hoisting var / let / const

Q03 this в разных контекстах

Q04 Arrow function vs regular function

Q05 Closure

Q06 Prototype chain

Q07 call / apply / bind

Q08 Spread vs rest

== VS ===**ОПИСАНИЕ**

== делает type coercion (1 == '1'). === – strict equality (без приведения). Always use === кроме сравнения с null.

КАК РАБОТАЕТ

- 01 == coerces.
- 02 === – type+value.
- 03 null == undefined – true (исключение).
- 04 NaN !== NaN.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review.
- 02 Linting (eqeqeq).

ПОДВОДНЫЕ КАМНИ

- 01 null check: x == null catches both null+undefined.
- 02 NaN: используй Number.isNaN.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Object.is?
- 02 Чем == null от === null отличается?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Equality_comparisons_and_sameness

ВОПРОС Q02

Hoisting var / let / const

ОПИСАНИЕ

var hoisted с initial undefined, let/const – TDZ. function declaration hoisted полностью.

КАК РАБОТАЕТ

- 01 var: declaration hoisted, init undefined.
- 02 let/const: TDZ до declaration.
- 03 function declaration: full hoist.
- 04 function expression: var-style hoist.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review.
- 02 Refactoring var → let.

ПОДВОДНЫЕ КАМНИ

- 01 TDZ ошибка: ReferenceError, не undefined.
- 02 Loop var bug – let решает.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое temporal dead zone?
- 02 Чем function declaration от expression отличается?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Glossary/Hoisting>

this в разных контекстах

ОПИСАНИЕ

Function this – global/undefined в strict. Method – instance. Arrow – lexical (внешний). new – instance.

КАК РАБОТАЕТ

- 01 Function call: global / undefined.
- 02 Method: object.
- 03 Arrow: lexical scope.
- 04 new: new instance.
- 05 bind/call/apply: explicit.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Class methods.
- 02 Event handlers.

ПОДВОДНЫЕ КАМНИ

- 01 Lost this в callback: () => bind.
- 02 Arrow в class field – auto-bound.

МОГУТ ДОСПРОСИТЬ

- 01 Чем arrow от function отличается?
- 02 Что такое bind?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this>

ВОПРОС Q04

Arrow function vs regular function

ОПИСАНИЕ

Arrow: lexical this, нет arguments, не constructor, не yield. Regular – есть всё.

КАК РАБОТАЕТ

- 01 Lexical this.
- 02 No arguments – use rest.
- 03 Cannot be new'd.
- 04 No prototype.
- 05 Implicit return.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Callbacks (preserve this).
- 02 Short utilities.

ПОДВОДНЫЕ КАМНИ

- 01 Class methods как arrow – нет prototype.
- 02 Object methods – теряют this auto-binding.

МОГУТ ДОСПРОСИТЬ

- 01 Когда arrow не подходит?
- 02 Что такое class field?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

Closure

ОПИСАНИЕ

Function которая запоминает переменные внешнего scope даже после возврата. Основа private state, currying, modules.

КАК РАБОТАЕТ

- 01 function returns function.
- 02 Inner refs outer vars.
- 03 Vars живут пока closure живёт.
- 04 Memory: closure держит refs.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Currying, memoization.
- 02 Module pattern.
- 03 Private state.

ПОДВОДНЫЕ КАМНИ

- 01 Memory leak – closure держит лишние refs.
- 02 Loop closure bug – let спасает.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое IIFE?
- 02 Чем closure от global отличается?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Closures>

ВОПРОС Q06

Prototype chain

ОПИСАНИЕ

Объекты наследуют через `[[Prototype]]`. Lookup по chain до `Object.prototype` → null.

КАК РАБОТАЕТ

- 01 `__proto__` / `Object.getPrototypeOf`.
- 02 `new` устанавливает `proto`.
- 03 `class` – sugar над `prototype`.
- 04 `hasOwnProperty` проверяет `own`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Inheritance`.
- 02 `Method sharing`.

ПОДВОДНЫЕ КАМНИ

- 01 Mutating `__proto__` – slow.
- 02 `for-in` iterates inherited.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `class`?
- 02 Чем `Object.create` от `new` отличается?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain

call / apply / bind

ОПИСАНИЕ

`call(thisArg, ...args)`. `apply(thisArg, [args])`. `bind` возвращает новую функцию с pre-bound `thisArg` + `args`.

КАК РАБОТАЕТ

- 01 `call` – invoke с `this+args`.
- 02 `apply` – то же, `args` как массив.
- 03 `bind` – return new fn.
- 04 Partial application через `bind`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `this fix`.
- 02 Borrowing methods.

ПОДВОДНЫЕ КАМНИ

- 01 `bind` создаёт новую функцию каждый вызов – perf.
- 02 Arrow ignore `call/apply/bind`.

МОГУТ ДОСПРОСИТЬ

- 01 Чем `bind` от `arrow` отличается?
- 02 Что такое soft `bind`?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/bind

ВОПРОС Q08

Spread vs rest

ОПИСАНИЕ

Spread (...): expand iterable. Rest (...args): collect into array. Same syntax, разные позиции.

КАК РАБОТАЕТ

- 01 Spread: `function(...arr), [...arr], {...obj}`.
- 02 Rest: `function(...args), const [a, ...rest]`.
- 03 Object rest/spread shallow copy.
- 04 ESNext: deep – нет.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Cloning.
- 02 Merging.
- 03 Function args.

ПОДВОДНЫЕ КАМНИ

- 01 Shallow copy – nested mutates.
- 02 Spread iterables (string), object – keys.

МОГУТ ДОСПРОСИТЬ

- 01 Чем spread от `Object.assign` отличается?
- 02 Что такое iterable protocol?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Spread_syntax

EVENT LOOP**PROMISE****MICROTASK****ASYNC**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

EVENT LOOP PROMISE MICROTASK ASYNC

ВОПРОСЫ ДНЯ

Q09 Event loop основы

Q10 Promise states

Q11 async / await

Q12 Microtask vs Macrotask

Q13 Promise.all vs allSettled vs race vs any

Q14 AbortController

Q15 setTimeout vs setInterval vs requestAnimationFrame

ВОПРОС Q09

Event loop основы

ОПИСАНИЕ

Stack + queue. Stack пуст → queue → callback → stack. Microtasks (Promise) перед macrotasks.

КАК РАБОТАЕТ

- 01 Call stack: sync.
- 02 Macrotask queue: setTimeout, fetch.
- 03 Microtask queue: Promise, queueMicrotask.
- 04 После каждого macrotask – все microtasks.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Async понимание.
- 02 Performance.

ПОДВОДНЫЕ КАМНИ

- 01 Long microtask блокирует UI.
- 02 setTimeout(0) не точно 0.

МОГУТ ДОСПРОСИТЬ

- 01 Чем microtask от macrotask отличается?
- 02 Что такое requestAnimationFrame?

ПОЛНАЯ СТАТЬЯ

<https://html.spec.whatwg.org/multipage/webappapis.html#event-loop>

Promise states

ОПИСАНИЕ

pending → fulfilled / rejected. Once settled – immutable. .then возвращает новый Promise.

КАК РАБОТАЕТ

```
01 .then(onFulfilled, onRejected).
02 .catch === .then(undef, fn).
03 .finally – без значения.
04 Chained Promise resolved value.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Async код.
```

ПОДВОДНЫЕ КАМНИ

```
01 Без catch – unhandled rejection.
02 return Promise в .then – flat chain.
```

МОГУТ ДОСПРОСИТЬ

```
01 Что такое Promise.race?
02 Чем allSettled полезен?
```

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

ВОПРОС Q11

async / await

ОПИСАНИЕ

Sugar над Promise. await pauses fn до resolve. try/catch для errors.

КАК РАБОТАЕТ

- 01 async fn returns Promise.
- 02 await unwraps.
- 03 try/catch для reject.
- 04 Sequential by default.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Аsync код.

ПОДВОДНЫЕ КАМНИ

- 01 Sequential await – slow вместо Promise.all.
- 02 await в loop – $n \cdot \text{latency}$.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое top-level await?
- 02 Чем async от Promise отличается?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function

Microtask vs Macrotask

ОПИСАНИЕ

Microtask: Promise, queueMicrotask. Macrotask: setTimeout, MessageChannel. Microtasks дренятся ПОСЛЕ каждой macrotask.

КАК РАБОТАЕТ

- 01 After each macrotask: drain ALL microtasks.
- 02 Microtask может добавить microtask – до пустоты.
- 03 Microtask приоритет выше.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Промисный flow.
- 02 Performance.

ПОДВОДНЫЕ КАМНИ

- 01 Long microtask chain блокирует rendering.
- 02 setTimeout(0) выполнится после microtasks.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое queueMicrotask?
- 02 Чем requestAnimationFrame от setTimeout отличается?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/HTML_DOM_API/Microtask_guide

ВОПРОС Q13

Promise.all vs allSettled vs race vs any

ОПИСАНИЕ

all: fast-fail. allSettled: всегда fulfill. race: первый settled. any: первый fulfilled.

КАК РАБОТАЕТ

- 01 all: array of values, reject on first.
- 02 allSettled: array of {status, value/reason}.
- 03 race: первый settled (fulfill or reject).
- 04 any: первый fulfill (или AggregateError).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Parallel async.
- 02 Timeout race.

ПОДВОДНЫЕ КАМНИ

- 01 all + один reject теряет всё.
- 02 race на reject – теряет slow success.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое AggregateError?
- 02 Чем all от allSettled отличается?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

AbortController

ОПИСАНИЕ

Cancel async-операции (fetch, ReadableStream). Сигнал через signal.

КАК РАБОТАЕТ

```
01 new AbortController().
02 fetch(url, {signal: ctl.signal}).
03 ctl.abort() → reject AbortError.
04 Listen abort event для cleanup.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Search cancel.
02 Component unmount.
```

ПОДВОДНЫЕ КАМНИ

```
01 Без abort – race condition (старый response перетирает новый).
02 AbortError – особый case в catch.
```

МОГУТ ДОСПРОСИТЬ

```
01 Что такое AbortSignal.timeout?
02 Чем cancel от Promise.cancel отличается?
```

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/AbortController>

ВОПРОС Q15

setTimeout vs setInterval vs requestAnimationFrame

ОПИСАНИЕ

setTimeout – once. setInterval – repeating (no wait). rAF – sync с monitor refresh.

КАК РАБОТАЕТ

- 01 setTimeout: minimal 4ms (nested).
- 02 setInterval: drift возможен.
- 03 rAF: 60Hz / monitor rate, pause в hidden tab.
- 04 Tab inactive – throttling.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Animations.
- 02 Polling.
- 03 Idle work.

ПОДВОДНЫЕ КАМНИ

- 01 setInterval не ждёт async – overlap.
- 02 setTimeout(0) не 0ms.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое requestIdleCallback?
- 02 Чем rAF от setTimeout отличается?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame>

REACT**KEYS****LIFTING****MEMO****USECALLBACK**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

REACT KEYS LIFTING MEMO USECALLBACK

ВОПРОСЫ ДНЯ

- Q16 Зачем key в списке?
- Q17 Lifting state up
- Q18 React.memo когда полезен
- Q19 useCallback когда полезен
- Q20 useMemo когда полезен
- Q21 Controlled vs Uncontrolled
- Q22 Что такое children prop?

ВОПРОС Q16

Зачем key в списке?

ОПИСАНИЕ

React reconciliation: связывает старые и новые element. Стабильный id → переупорядочивание, не пересоздание.

КАК РАБОТАЕТ

- 01 Stable id (db id).
- 02 Не index если порядок меняется.
- 03 Внутри array только.
- 04 Уникальный среди sibling.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Lists, tables.

ПОДВОДНЫЕ КАМНИ

- 01 Index как key + reorder = lose state.
- 02 Math.random() – fresh each render.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое reconciliation?
- 02 Чем index-key от id-key отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/rendering-lists>

Lifting state up

ОПИСАНИЕ

Когда два sibling нужно sync – поднимай state в общий parent. Передавай callback.

КАК РАБОТАЕТ

- 01 State в LCA (lowest common ancestor).
- 02 Передай value + onChange как props.
- 03 Children – controlled.
- 04 Альтернатива: Context, store.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Form-связи.
- 02 Filter + list.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком высоко – prop drilling.
- 02 Слишком низко – desync.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое prop drilling?
- 02 Чем Context от store отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/sharing-state-between-components>

ВОПРОС Q18

React.memo когда полезен

ОПИСАНИЕ

Когда: parent часто рендерится, props стабильны, child тяжёлый. Иначе – overhead.

КАК РАБОТАЕТ

- 01 React.memo(Component).
- 02 Shallow compare props.
- 03 Custom areEqual для deep.
- 04 Profile перед добавлением.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Heavy children.
- 02 List items.

ПОДВОДНЫЕ КАМНИ

- 01 Без memoize props (objects, fns) – бесполезно.
- 02 Premature memo – overhead.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое useMemo?
- 02 Чем PureComponent от мемо отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/memo>

useCallback когда полезен

ОПИСАНИЕ

Когда передаёшь callback в мемо'd child или в deps другого hook. Иначе – useless.

КАК РАБОТАЕТ

- 01 `useCallback(fn, deps).`
- 02 Returns same fn между rendersa если deps равны.
- 03 Полезен для мемо'd children и hook deps.
- 04 Без мемо child – overhead.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 мемо'd handlers.
- 02 Effect/мемо deps.

ПОДВОДНЫЕ КАМНИ

- 01 Stale closure – wrong deps.
- 02 `useCallback` везде – premature.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `useMemo`?
- 02 Чем `useCallback` от `useRef` отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useCallback>

ВОПРОС Q20

useMemo когда полезен

ОПИСАНИЕ

Дорогое вычисление от deps. Memoize – пропустить если deps не менялись.

КАК РАБОТАЕТ

- 01 useMemo(() => compute(), deps).
- 02 Skip compute если deps равны.
- 03 Profile сначала.
- 04 NOT для side effects.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Heavy computation.
- 02 Object identity для memo'd children.

ПОДВОДНЫЕ КАМНИ

- 01 useMemo для primitives – overhead.
- 02 Wrong deps – stale.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое useDeferredValue?
- 02 Чем useMemo от React.memo отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useMemo>

Controlled vs Uncontrolled

ОПИСАНИЕ

Controlled: state в React (value + onChange). Uncontrolled: ref + defaultValue.

КАК РАБОТАЕТ

- 01 Controlled: value+onChange.
 - 02 Uncontrolled: defaultValue+ref.
 - 03 Controlled – single source of truth.
 - 04 Uncontrolled – простой код для simple forms.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Forms.
-

ПОДВОДНЫЕ КАМНИ

- 01 Controlled – каждый keystroke render.
 - 02 Uncontrolled – react-hook-form-style.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое react-hook-form?
 - 02 Чем defaultValue от value отличается?
-

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/sharing-state-between-components>

ВОПРОС Q 2 2

Что такое children prop?

ОПИСАНИЕ

Special prop – содержимое между tags. Позволяет composition.

КАК РАБОТАЕТ

- 01 `<Card>content</Card>` → `props.children = 'content'`.
- 02 Может быть string, element, array.
- 03 `React.Children` API для iterate.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Layouts.
- 02 Wrappers.
- 03 Slots.

ПОДВОДНЫЕ КАМНИ

- 01 map по children без `React.Children` – break на arrays.
- 02 children typing – `ReactNode`.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое render props?
- 02 Чем children от render-prop отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/passing-props-to-a-component#passing-jsx-as-children>

HOOKS**USEEFFECT****USEREF****CONTEXT**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

HOOKS USEEFFECT USEREF CONTEXT

ВОПРОСЫ ДНЯ

Q23 useEffect vs useLayoutEffect

Q24 Cleanup в useEffect

Q25 Dependency array

Q26 useRef vs useState

Q27 forwardRef

Q28 Context – когда и зачем

Q29 useReducer vs useState

ВОПРОС Q23

useEffect vs useEffect

ОПИСАНИЕ

useEffect: async после paint. useEffect: sync перед paint. Layout – для DOM measurement.

КАК РАБОТАЕТ

- 01 useEffect: async, не блокирует paint.
- 02 useEffect: sync, перед paint.
- 03 Layout для DOM read.
- 04 SSR: useEffect – warning.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 DOM measure.
- 02 Animation triggers.

ПОДВОДНЫЕ КАМНИ

- 01 useEffect блокирует – slow.
- 02 SSR mismatch warning.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое useInsertionEffect?
- 02 Чем useEffect от useEffect отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useLayoutEffect>

Cleanup в useEffect

ОПИСАНИЕ

Return функция вызывается перед next effect и при unmount. Cancel subscriptions, abort, clearTimeout.

КАК РАБОТАЕТ

- 01 return () => cleanup.
- 02 Runs перед next effect и unmount.
- 03 clearTimeout, removeEventListener, abort.
- 04 Без cleanup – leak / race.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Subscriptions.
- 02 Timers.
- 03 Fetches.

ПОДВОДНЫЕ КАМНИ

- 01 Без cleanup – multiple listeners.
- 02 StrictMode: effect runs twice – cleanup тестит.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое StrictMode?
- 02 Чем clearInterval от clearTimeout отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useEffect>

ВОПРОС Q25

Dependency array

ОПИСАНИЕ

Empty []: один раз. Без массива: каждый render. С deps: при изменении.

КАК РАБОТАЕТ

- 01 [] – mount только.
- 02 Без deps – every render.
- 03 [a, b] – when a/b change.
- 04 Lint exhaustive-deps подскажет.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Subscriptions.
- 02 Fetches.

ПОДВОДНЫЕ КАМНИ

- 01 Wrong deps – stale closure.
- 02 Object/fn в deps без memoize – каждый render.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое exhaustive-deps?
- 02 Как избегать race condition?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useState>

useRef vs useState

ОПИСАНИЕ

useRef: mutable container, не triggers render. useState: triggers render.

КАК РАБОТАЕТ

- 01 useRef.current – mutable.
 - 02 Не вызывает re-render.
 - 03 Для DOM ref, mutable values.
 - 04 useState для UI state.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 DOM access.
 - 02 Imperative APIs.
 - 03 Mutable counters.
-

ПОДВОДНЫЕ КАМНИ

- 01 Don't read ref.current во время render – race.
 - 02 useRef для derived state – anti-pattern.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое forwardRef?
 - 02 Чем useRef от createRef отличается?
-

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useRef>

ВОПРОС Q 27

forwardRef

ОПИСАНИЕ

Прокидывает ref через component. Без forwardRef ref на functional component – undefined.

КАК РАБОТАЕТ

- 01 `React.forwardRef((props, ref) => ...).`
- 02 Передавай ref в DOM или другой forwardRef.
- 03 `useImperativeHandle` для custom API.
- 04 React 19+ ref как regular prop.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Reusable inputs.
- 02 UI library components.

ПОДВОДНЫЕ КАМНИ

- 01 `forgetForwardRef` – ref undefined warning.
- 02 `useImperativeHandle` – escape hatch.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `useImperativeHandle`?
- 02 Чем ref от prop отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/forwardRef>

Context – когда и зачем

ОПИСАНИЕ

Глобальный state без prop drilling. Theme, locale, auth, currentUser.

КАК РАБОТАЕТ

- 01 createContext + Provider + useContext.
 - 02 Re-render всех consumers при изменении value.
 - 03 Memoize value object.
 - 04 Split contexts для гранулярности.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Theme.
 - 02 Auth.
 - 03 Locale.
-

ПОДВОДНЫЕ КАМНИ

- 01 Update value – все consumers re-render.
 - 02 Объект в value без мемо – каждый render.
-

МОГУТ ДОСПРОСИТЬ

- 01 Чем Context от Redux отличается?
 - 02 Что такое context selector?
-

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useContext>

ВОПРОС Q29

useReducer vs useState

ОПИСАНИЕ

useReducer для сложного state (multiple fields, transitions). useState – простые primitives.

КАК РАБОТАЕТ

- 01 useReducer(reducer, initial).
- 02 dispatch(action).
- 03 Reducer pure (state, action) → state.
- 04 Лучше тестируется.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Forms.
- 02 State machines.
- 03 Complex UI state.

ПОДВОДНЫЕ КАМНИ

- 01 Простой counter – overkill.
- 02 Mutation в reducer – anti-pattern.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое immer?
- 02 Чем useReducer от Redux отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useReducer>

BROWSER**CORS****COOKIE****STORAGE****CSP**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

BROWSER CORS COOKIE STORAGE CSP

ВОПРОСЫ ДНЯ

- Q30 CORS — что это и как работает
- Q31 Cookie атрибуты — Secure, HttpOnly, SameSite
- Q32 localStorage vs sessionStorage vs cookie
- Q33 CSP — что и зачем
- Q34 XSS — типы и защита
- Q35 CSRF — что и защита
- Q36 Service Worker — для чего

ВОПРОС Q30

CORS – что это и как работает

ОПИСАНИЕ

Cross-Origin Resource Sharing. Browser блокирует cross-origin запросы; сервер должен разрешить через Access-Control-Allow-Origin.

КАК РАБОТАЕТ

- 01 Same-origin policy.
- 02 Server отвечает Access-Control-Allow-* headers.
- 03 Preflight OPTIONS для non-simple.
- 04 Credentials отдельно.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 API запросы на other domain.

ПОДВОДНЫЕ КАМНИ

- 01 * + credentials – server error.
- 02 CORS errors в console – не network failure.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое preflight?
- 02 Чем CORS от CSP отличается?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

Cookie атрибуты – Secure, HttpOnly, SameSite

ОПИСАНИЕ

Secure: только HTTPS. HttpOnly: JS не видит. SameSite: ограничивает cross-site.

КАК РАБОТАЕТ

- 01 Secure: HTTPS only.
 - 02 HttpOnly: document.cookie не видит.
 - 03 SameSite=Lax/Strict/None.
 - 04 Domain + Path scope.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Auth.
 - 02 Session.
-

ПОДВОДНЫЕ КАМНИ

- 01 SameSite=None требует Secure.
 - 02 Без HttpOnly – XSS читает session.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое CSRF?
 - 02 Чем localStorage от cookie отличается?
-

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>

ВОПРОС Q32

localStorage vs sessionStorage vs cookie

ОПИСАНИЕ

localStorage: persistent, ~5MB, sync API. sessionStorage: per-tab. cookie: сервер видит, маленький.

КАК РАБОТАЕТ

- 01 localStorage: до clear.
- 02 sessionStorage: до tab close.
- 03 cookie: server-readable, 4KB.
- 04 JSON serialize вручную.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 UI prefs.
- 02 Auth tokens (cookie лучше).

ПОДВОДНЫЕ КАМНИ

- 01 localStorage sync – блокирует main.
- 02 XSS читает localStorage.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое IndexedDB?
- 02 Чем cookie от localStorage отличается?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

CSP – что и зачем

ОПИСАНИЕ

Content Security Policy: HTTP-header ограничивающий sources for scripts/styles/images. Защита от XSS.

КАК РАБОТАЕТ

- 01 Content-Security-Policy header.
- 02 script-src 'self' cdn.com.
- 03 Inline scripts требуют 'unsafe-inline' или nonce.
- 04 report-uri для violations.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 XSS mitigation.
- 02 Production sites.

ПОДВОДНЫЕ КАМНИ

- 01 unsafe-inline – теряет смысл.
- 02 CSP сложно отлаживать без report.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое nonce vs hash?
- 02 Чем CSP от CORS отличается?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>

ВОПРОС Q34

XSS – типы и защита

ОПИСАНИЕ

Stored, Reflected, DOM-based. Защита: escape user input, CSP, HttpOnly cookies, sanitize HTML (DOMPurify).

КАК РАБОТАЕТ

- 01 Stored: persisted в DB.
- 02 Reflected: query params.
- 03 DOM-based: client-side template.
- 04 Sanitize + escape + CSP.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 User-generated content.

ПОДВОДНЫЕ КАМНИ

- 01 dangerouslySetInnerHTML без sanitize.
- 02 React escapes по умолчанию – но не атрибуты полностью.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое DOMPurify?
- 02 Чем XSS от CSRF отличается?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-community/attacks/xss/>

CSRF – что и защита

ОПИСАНИЕ

Cross-Site Request Forgery: жертва на attacker-сайте, browser шлёт cookie на target.
Защита: SameSite cookie, CSRF-token, double-submit.

КАК РАБОТАЕТ

- 01 Browser auto-attaches cookie.
- 02 Defense: SameSite=Lax/Strict.
- 03 CSRF-token в form (hidden field).
- 04 Double-submit pattern.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 State-changing requests.

ПОДВОДНЫЕ КАМНИ

- 01 GET для mutations – auto vulnerable.
- 02 SameSite=None без Secure – invalid.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое SameSite?
- 02 Чем CSRF от XSS отличается?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-community/attacks/csrf>

ВОПРОС Q36

Service Worker – для чего

ОПИСАНИЕ

Прокси между app и network. Offline cache, push notifications, background sync.

КАК РАБОТАЕТ

- 01 `register('/sw.js')`.
- 02 `fetch` event – intercept.
- 03 `cache-first / network-first`.
- 04 Update lifecycle: `install`, `activate`, `skipWaiting`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 PWA.
- 02 Offline.
- 03 Push notifications.

ПОДВОДНЫЕ КАМНИ

- 01 Update reload – нужен `skipWaiting` + `claim`.
- 02 Cache stale без `versioning`.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Workbox?
- 02 Чем PWA от native отличается?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

PERF LCP INP CLS BUNDLE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

PERF LCP INP CLS BUNDLE

ВОПРОСЫ ДНЯ

- Q37 LCP — что измеряет, целевые значения
- Q38 INP — что измеряет
- Q39 CLS — что измеряет
- Q40 Code splitting
- Q41 Tree shaking
- Q42 Lazy loading images
- Q43 Bundle analysis

ВОПРОС Q37

LCP – что измеряет, целевые значения

ОПИСАНИЕ

Largest Contentful Paint: время до отображения largest above-fold element. Good < 2.5s, poor > 4s.

КАК РАБОТАЕТ

- 01 Hero image, large text.
- 02 Influenced by server, render-blocking, image size.
- 03 Optimize: preload, CDN, format (AVIF/WebP), priority hint.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Web Vitals.

ПОДВОДНЫЕ КАМНИ

- 01 Lazy hero image – bad LCP.
- 02 Late font swap – LCP shift.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое FCP?
- 02 Чем LCP от FID отличается?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/lcp>

INP – что измеряет

ОПИСАНИЕ

Interaction to Next Paint: latency от user input до next paint. Replaces FID. Good < 200ms.

КАК РАБОТАЕТ

- 01 Все interactions: click, tap, key.
- 02 Worst-case (98th percentile).
- 03 Long tasks повышают INP.
- 04 Optimize: split JS, defer, react concurrent.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Web Vitals.

ПОДВОДНЫЕ КАМНИ

- 01 Long handlers – высокий INP.
- 02 Без yield – block UI.

МОГУТ ДОСПРОСИТЬ

- 01 Чем INP от FID отличается?
- 02 Что такое long task?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/inp>

ВОПРОС Q39

CLS – что измеряет

ОПИСАНИЕ

Cumulative Layout Shift: сумма unexpected layout shifts. Good < 0.1.

КАК РАБОТАЕТ

- 01 Image без width/height.
- 02 Late-loading fonts.
- 03 Injected ads / banners.
- 04 Use min-height, font-display, reserve space.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Web Vitals.

ПОДВОДНЫЕ КАМНИ

- 01 Aspect-ratio для images.
- 02 Late-loading content без reserved space.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое font-display: swap?
- 02 Чем CLS от LCP отличается?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/cls>

Code splitting

ОПИСАНИЕ

Bundle разбит на chunks загружаемые по требованию. `React.lazy` + `Suspense` / `dynamic import`.

КАК РАБОТАЕТ

- 01 `import('./mod')` возвращает `Promise`.
- 02 Webpack/Vite split автоматически.
- 03 `React.lazy(() => import)`.
- 04 `Suspense fallback`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Heavy routes.
- 02 Modals.
- 03 Admin sections.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком много chunks – overhead.
- 02 Split не по route – bad UX.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое route-based splitting?
- 02 Чем `dynamic import` от `require` отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/lazy>

ВОПРОС Q41

Tree shaking

ОПИСАНИЕ

Bundler удаляет unused exports. Требуется ESM (import/export).

КАК РАБОТАЕТ

- 01 ESM static analysis.
- 02 CommonJS – не shake-able.
- 03 sideEffects: false в package.json.
- 04 Pure annotations: /*#__PURE__*/.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Library imports.
- 02 Reduce bundle.

ПОДВОДНЫЕ КАМНИ

- 01 Side-effect imports – не shake.
- 02 CJS deps в bundle.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое sideEffects: false?
- 02 Чем tree-shaking от dead code elimination отличается?

ПОЛНАЯ СТАТЬЯ

<https://webpack.js.org/guides/tree-shaking/>

Lazy loading images

ОПИСАНИЕ

loading='lazy' на `img`. Native в современных browsers. Грузится при approach viewport.

КАК РАБОТАЕТ

- 01 `<img loading='lazy'>`.
 - 02 `decoding='async'`.
 - 03 Aspect-ratio для CLS.
 - 04 `fetchpriority='high'` для LCP.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Long pages.
 - 02 Galleries.
-

ПОДВОДНЫЕ КАМНИ

- 01 Lazy hero – bad LCP – eager + priority hint.
 - 02 Без size – CLS.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `fetchpriority`?
 - 02 Чем native от `IntersectionObserver` lazy отличается?
-

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/lazy-loading-images>

ВОПРОС Q43

Bundle analysis

ОПИСАНИЕ

Webpack-bundle-analyzer / vite-bundle-visualizer. Видишь size каждого chunk и dependency.

КАК РАБОТАЕТ

- 01 Stats.json + analyzer.
- 02 Treemap visualisation.
- 03 Source-map-explorer для published.
- 04 CI: assert size budget.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Performance audit.

ПОДВОДНЫЕ КАМНИ

- 01 Без budgets – drift.
- 02 Polyfills double-shipped.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое source-map-explorer?
- 02 Чем performance budget полезен?

ПОЛНАЯ СТАТЬЯ

<https://github.com/webpack-contrib/webpack-bundle-analyzer>

TS GIT TOOLS A11Y

ЧТО УЧИМ В ЭТОТ ДЕНЬ

TS GIT TOOLS A11Y

ВОПРОСЫ ДНЯ

- Q44 TypeScript: any vs unknown
- Q45 TypeScript: type vs interface
- Q46 TypeScript: generics
- Q47 git rebase vs merge
- Q48 Webpack vs Vite
- Q49 Semantic HTML — почему важно
- Q50 ARIA — когда использовать

В О П Р О С Q 4 4

TypeScript: any vs unknown

ОПИСАНИЕ

any: opt-out. unknown: type-safe wildcard – нужен narrow перед use.

КАК РАБОТАЕТ

- 01 any – disables checks.
- 02 unknown – нельзя использовать без guard.
- 03 typeof / instanceof / custom guards.
- 04 Prefer unknown.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 External JSON.
- 02 Catch parameter.

ПОДВОДНЫЕ КАМНИ

- 01 any распространяется – silent bugs.
- 02 unknown заставляет narrow.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое type guard?
- 02 Чем never от unknown отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#unknown>

TypeScript: type vs interface

ОПИСАНИЕ

Оба describe shape. interface – extendable, declaration merging. type – unions, intersections, computed.

КАК РАБОТАЕТ

- 01 interface: extends, merging.
- 02 type: union, intersect, computed.
- 03 interface для public API.
- 04 type для unions, mappings.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 DTO.
- 02 Component props.

ПОДВОДНЫЕ КАМНИ

- 01 Declaration merging – surprise.
- 02 Сложные types через interface – не работают.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое declaration merging?
- 02 Чем type от interface отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#type-aliases>

ВОПРОС Q 46

TypeScript: generics

ОПИСАНИЕ

Параметризованные types. `function<T>(x: T): T`. Полезно для reusable utilities.

КАК РАБОТАЕТ

- 01 `<T>` – параметр.
- 02 `extends` – constraint.
- 03 Default: `<T = string>`.
- 04 Variance: covariant, contravariant.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Library APIs.
- 02 Hooks (`useState<T>`).

ПОДВОДНЫЕ КАМНИ

- 01 Слишком много T – нечитабельно.
- 02 Inference часто лучше manual.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `infer`?
- 02 Чем variance важна?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/generics.html>

git rebase vs merge

ОПИСАНИЕ

rebase: переписывает history линейно. merge: создаёт merge commit. rebase для feature-branches.

КАК РАБОТАЕТ

- 01 rebase: replay commits.
- 02 merge: merge commit.
- 03 rebase + force-push если уже pushed.
- 04 Squash для clean history.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Feature workflow.
- 02 Open PRs.

ПОДВОДНЫЕ КАМНИ

- 01 rebase shared branch – chaos.
- 02 force-push без --force-with-lease – danger.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое --force-with-lease?
- 02 Чем squash от rebase отличается?

ПОЛНАЯ СТАТЬЯ

<https://git-scm.com/book/en/v2/Git-Branching-Rebasing>

В О П Р О С Q 4 8

Webpack vs Vite

ОПИСАНИЕ

Webpack: bundler, slow dev. Vite: ESM dev (instant HMR), Rollup для prod.

КАК РАБОТАЕТ

- 01 Vite dev: native ESM, no bundle.
- 02 Vite prod: Rollup.
- 03 Webpack: full bundle dev.
- 04 HMR: instant в Vite.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Modern apps.

ПОДВОДНЫЕ КАМНИ

- 01 Vite не для CommonJS-heavy.
- 02 Plugin ecosystem меньше Webpack.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое esbuild?
- 02 Чем Rollup от Webpack отличается?

ПОЛНАЯ СТАТЬЯ

<https://vitejs.dev/>

Semantic HTML – почему важно

ОПИСАНИЕ

Tags несут смысл (article, nav, button). Screen readers, SEO, default behavior.

КАК РАБОТАЕТ

- 01 <button> auto keyboard, role.
 - 02 <nav>, <main>, <article> – landmarks.
 - 03 <a> для navigation.
 - 04 <form> – submit поведение.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 A11y.
 - 02 SEO.
-

ПОДВОДНЫЕ КАМНИ

- 01 <div onClick> – клавиатура broken.
 - 02 – bad.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое landmark?
 - 02 Чем role от semantic tag отличается?
-

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Glossary/Semantics>

ВОПРОС Q50

ARIA – когда использовать

ОПИСАНИЕ

Когда semantic HTML недостаточно. Disable если можешь использовать native (button, a).

КАК РАБОТАЕТ

- 01 No ARIA > bad ARIA.
- 02 aria-label для no-text buttons.
- 03 aria-live для dynamic content.
- 04 role=' для custom widgets.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Custom dropdown.
- 02 Modal.

ПОДВОДНЫЕ КАМНИ

- 01 aria-hidden на focusable – broken.
- 02 role mismatch с element.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое APG?
- 02 Чем role от semantic отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.w3.org/WAI/ARIA/apg/>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ JS core: ===, this, arrow, closure, prototype
- ☐ Async: event loop, Promise, await, AbortController
- ☐ React: keys, memo, useCallback, controlled vs uncontrolled
- ☐ Hooks: useEffect, useRef, Context, useReducer
- ☐ Browser: CORS, Cookie attributes, CSP, XSS, CSRF
- ☐ Performance: LCP, INP, CLS, code splitting, tree shaking
- ☐ TS: any vs unknown, type vs interface, generics
- ☐ Tools: webpack vs vite, git rebase, semantic HTML, ARIA

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru